

Objects 4

Brainster Web Development Academy



Table of contents	01	The “prototype” property
	02	Inheritance: a bit of theory
	03	Prototype inheritance
	04	Class inheritance
	05	The big reveal
	06	FP vs OOP in JavaScript



Exercise 1

1. Create a Person object prototype
2. The constructor function should take two input arguments - firstName and lastName. Set those to the prototype (this.firstName = firstName ...)
3. Instantiate a new person (e.g. let person1 = new Person('Harry', 'Dresden');



The “prototype” property

- We already discussed what object prototypes were - functions that produce objects.
- We can think of them as "factories" that help us create objects, by using the "new" keyword.
- But what if we needed to do something with the object prototype after its declaration? Say, we want to add an extra method or property? Or maybe reuse those methods / properties somewhere else?

3

```
function Person(name) {  
    this.name = name;  
  
    this.sayName = function() {  
        alert(this.name);  
    }  
}
```

```
const person1 = new Person('Harry'); // A regular object  
console.log(Person.prototype); // The prototype we build from
```



The “prototype” property

- If we can access it, we can modify it (that includes the **constructor** too).
- For example, we'd like to add a greeting method:

```
Person.prototype.greet = function() {  
    alert('Hello!');  
}
```

- This means that **every existing** and **new** Person will also get a greet method!



The “prototype” property

- One interesting thing to note here is the behavior of “**this**”.
- When in the context of a prototype (or a class, but we'll get to that later), it retains its meaning:

```
Person.prototype.greetWithName = function() {  
    alert('Hello, my name is ' + this.name);  
}
```

3

- “**this**” behaves in the same way as if we would have defined it inside the body of an object prototype.



Exercise 2

Extend exercise 1:

1. After instantiating a person, add a new method to the Person object prototype: sayFullName, which simply alert the first and last name of a person instance.
2. Invoke the new method on the existing instance.



Inheritance: a bit of theory

- In programming, put in the simplest terms, inheritance is a way of "transferring" methods and properties from a parent to a child object.
- We can use the **.call** method for exactly this purpose - We can use the .call method for exactly this purpose - it allows us to re-use an existing object prototype as the parent.
- JavaScript classes also have a way of achieving this, but more on that later.



Prototype inheritance

```
function Person(name) {  
  this.name = name;  
  
  this.introduce = function() {  
    alert('Hello, my name is ' + this.name);  
  }  
}  
  
const p = new Person('Harry');  
p.introduce();  
console.log(p);
```

```
function Doctor(name, experience) {  
  Person.call(this, name);  
  this.experience = experience;  
}  
  
const d = new Doctor('Sally', 15);  
d.introduce();  
console.log(d);
```



Exercise 3

Extend exercise 2:

1. Create a new object prototype called Programmer. In addition to receiving firstName and lastName as input arguments, it should also receive a favoriteLanguage argument.
2. It also should **inherit** the prototype from the Person object prototype - that means, it should have a sayFullName method without it explicitly being defined in the Programmer object prototype (see slide 12 for a hint).
3. Add a new method to the Programmer object prototype: sayFavoriteLanguage, which should alert their favoriteLanguage.
4. Create a new Programmer, and call the sayFullName and sayFavoriteLanguage methods.



Class inheritance

- In JavaScript, class inheritance isn't too different from object prototypes for what it does.
- However, syntactically some people find it more readable - it's more obvious what comes from where.
- In the end, it's just a matter of preference (so neither is the right or wrong way - use what feels comfortable).



Class inheritance

```
class Person {  
    constructor(name) {  
        this.name = name;  
    }  
  
    introduce() {  
        alert('Hello, my name is ' + this.name);  
    }  
}  
  
const p = new Person('Harry');  
p.introduce();  
console.log(p);
```

```
class Doctor extends Person {  
    constructor(name, experience) {  
        super(name);  
        this.experience = experience;  
    }  
}  
  
const d = new Doctor('Sally', 15);  
d.introduce();  
console.log(d);
```



Class inheritance

There are two differences (aside from formatting):

- The use of the **extends** keyword to signify what's our parent class
- The use of **super** to pass constructor arguments to the parent class (similar to how we set the .prototype with object prototypes).



Exercise 4

1. Rewrite exercise III using classes. Same rules, same inheritance, but use classes instead of object prototypes.



The big reveal

- Classes and object prototypes are **the same thing** in JavaScript.
- Classes were introduced to the language later, but internally they are build on top of object prototypes.
- They are interoperable! In other words, you can **extend** an object prototype using a class (and vice versa).
- While both are useful, you can write an application without using either - plain objects are often more than enough. Keep it simple.



Exercise 5

Let's rewrite the code from the previous lecture and homework (the dog and cat one):

1. Create an empty array called "dogs".
2. Define an Animal class (or object prototype), which has a name and color property.
3. Make a Dog class (or object prototype) that extends the Animal class.
4. (In html) create two input fields (with different ids) and a button that says "Add dog".
5. Create a function which:
 - a. Gets the values from the two input fields
 - b. Creates a new "dog" **instance** using the first value as name and the second as color.
 - c. Adds the newly created dog object to the array.
 - d. Adds the newly created dog in an HTML table.
 - e. Logs in the console the whole "dogs" array
6. Invoke the function when the button gets clicked (using "click" with addEventListener)
7. The dog class should also have an extra property called animalType, which equals to 'dog'.
8. Whenever a new animal is entered, clear the value from the inputs.



Exercise 6

Extend the previous exercise:

1. Define an extra animal type (Cat, Bird...) which extends Animal. Just like Dog, it should also have an `animalType` property which has the value of 'cat' or 'bird', as appropriate.
2. Add a dropdown next to the text inputs which contains all of the possible animal types. (also in the homework, so if you did it, use that)
3. When a new animal is created (Dog, Bird, Cat...) create an instance of the appropriate type.



Exercise 7

Extend the previous exercise:

1. Add an extra column to the animals table - 'type', which simply prints out the animalType property, regardless of the instance type (they all have it anyway!)



Functional vs Object-Oriented Programming in Javascript

- JavaScript is a versatile language that supports both functional and object-oriented programming paradigms.
- Although you can use both, JavaScript is becoming more and more a functional programming language.
- Let's look at the main differences between FP and OOP.



Functional programming in JavaScript

7

- **Definition:** In FP, functions are first-class citizens, which means they can be passed as arguments, returned from other functions, and assigned to variables.
- **Immutability:** One of the core principles of FP is immutability. Instead of changing the existing data, new data structures are created.
- **Pure Functions:** These are functions that always produce the same output given the same input and have no side effects.
- **Higher-order Functions:** Functions that take other functions as arguments or return functions. Examples in JavaScript include map, filter, and reduce.
- **Statelessness:** In FP, functions do not maintain state. They operate only on the input provided to them.
- **Examples:** Using array methods like map, filter, and reduce are common FP patterns in JavaScript.



Object Oriented programming in JavaScript

7

- **Definition:** OOP is a paradigm based on the concept of "objects", which can contain data and code: data in the form of fields (often known as attributes or properties), and code, in the form of methods.
- **Classes and Objects:** A class is a blueprint for creating objects. An object is an instance of a class.
- **Inheritance:** Classes can inherit properties and methods from other classes, promoting code reusability.
- **Encapsulation:** The bundling of data and methods that operate on that data, restricting the direct access to some of the object's components. This can be achieved in JavaScript using closures, symbols, or the newer class syntax with private fields.
- **Polymorphism:** A principle that allows objects of different classes to be treated as objects of a common superclass. This can be achieved in JavaScript through prototypes or class inheritance.
- **Examples:** The introduction of the class keyword in ES6 made OOP more explicit in JavaScript, allowing for class declarations, constructors, and method definitions.



Object Oriented programming in JavaScript

7

- **Definition:** OOP is a paradigm based on the concept of "objects", which can contain data and code: data in the form of fields (often known as attributes or properties), and code, in the form of methods.
- **Classes and Objects:** A class is a blueprint for creating objects. An object is an instance of a class.
- **Inheritance:** Classes can inherit properties and methods from other classes, promoting code reusability.
- **Encapsulation:** The bundling of data and methods that operate on that data, restricting the direct access to some of the object's components. This can be achieved in JavaScript using closures, symbols, or the newer class syntax with private fields.
- **Polymorphism:** A principle that allows objects of different classes to be treated as objects of a common superclass. This can be achieved in JavaScript through prototypes or class inheritance.
- **Examples:** The introduction of the class keyword in ES6 made OOP more explicit in JavaScript, allowing for class declarations, constructors, and method definitions.



Object Oriented programming in JavaScript

- Historically, JavaScript was more functionally-inclined due to its prototype-based nature. However, with the introduction of ES6 (ECMAScript 2015) and subsequent updates, the language has introduced more OOP features, such as classes, while still retaining its functional characteristics.
- This flexibility allows developers to use a mix of both paradigms, often termed as multi-paradigm programming, enabling them to leverage the strengths of both.
- In conclusion, whether using functional or object-oriented programming in JavaScript, the choice largely depends on the problem being solved, the design preferences of the developer, and sometimes the requirements of the project.

